
MetaDEX03 Solo Audit Report

Prepared by Riley Holterhus

August 29, 2026

Contents

1	Introduction	3
1.1	About MetaDEX03	3
1.2	About the Auditor	3
1.3	Disclaimer	3
2	Audit Overview	4
2.1	Scope of Work	4
2.2	Summary of Findings	8
3	Findings	9
3.1	High Severity Findings	9
3.1.1	Try/catch blocks can be made to intentionally fail	9
3.1.2	Dispatch refundRecipient enables reentrancy concerns	9
3.2	Medium Severity Findings	10
3.2.1	Unrelated Relay balance changes could satisfy swap-output checks	10
3.2.2	Failed balance reads could corrupt balance-diff accounting	10
3.2.3	Per-batch reward rounding could leave the Relay unable to pay claims	11
3.3	Low Severity Findings	11
3.3.1	Permissionless index updates could delay migration fee flushing	11
3.3.2	Withdrawn token IDs could not be revived	12
3.3.3	Gauge fee tokens could differ from the underlying pool tokens	12
3.3.4	Relay reward configuration could disable reward processing	12
3.3.5	Invalid module return data could still revert pool swaps	13

1 Introduction

1.1 About MetaDEX03

MetaDEX03 is Dromos Labs' latest evolution of the liquidity coordination architecture used in Aerodrome and Velodrome. It retains the same core model of liquidity provision, voting, gauges, and emissions, while substantially reworking how those components interact. Several processes that previously operated around discrete weekly epochs have been redesigned to update more continuously as protocol state changes.

The protocol also introduces a multi-chain architecture, with Base serving as the root chain and supported leaf chains communicating through Hyperlane. Alongside these changes, MetaDEX03 introduces a new canonical AERO token and migration path from the previous protocols, as well as new components such as the Metarouter, Token Registry, and modular AMM fee and hook infrastructure.

1.2 About the Auditor

Riley Holterhus is an independent security researcher who focuses on Solidity smart contracts. Other than conducting independent security reviews, he searches for vulnerabilities in live codebases, and provides broader security consulting to protocols. Riley can be reached by email at rileyholterhus@gmail.com, by Telegram at [@holterhus](https://t.me/holterhus) and on Twitter/X at [@rileyholterhus](https://twitter.com/rileyholterhus).

1.3 Disclaimer

This report is intended to detail the identified vulnerabilities of the reviewed smart contracts and should not be construed as an endorsement or recommendation of any project, individual or entity. While the authors have made reasonable efforts to detect potential issues, the absence of any undetected vulnerabilities or issues cannot be guaranteed. Additionally, the security of the smart contracts may be affected by future changes or updates. By using the information in this report, you acknowledge that you are doing so at your own risk and that you should exercise your own judgment when implementing any recommendations or making decisions based on the findings. This report has been provided on an "as-is" basis and DOES NOT CONSTITUTE A GUARANTEE OR WARRANTY OF ANY FORM.

2 Audit Overview

2.1 Scope of Work

During June, July, and August 2026, Riley Holterhus provided security support to the Dromos team during the development of the MetaDEX03 protocol. This work included helping plan and coordinate security reviews ahead of launch, including an audit with ChainSecurity, a Sherlock Blackthorn review, and a Sherlock audit contest beginning in late August 2026. Riley also directly reviewed the codebase and identified potential security issues, which are documented in this report.

Given the overlap in review scope across the different security efforts, some issues relevant to Riley's review may also be covered in the other audit reports. ***To avoid repetition, this report focuses on findings not already documented by the other teams.***

The MetaDEX03 codebase evolved throughout the review period, and there was no single frozen commit that represented the full scope of Riley's review. Incoming PRs and other changes were reviewed as they were introduced. The relevant repos are: [dromos-labs/metadex](#) and [dromos-labs/metadex-slipstream](#). By the end of the review period, the agreed-upon scope consisted of:

```
metadex/V3/src/  
  
access/  
  GuardedAccessControlEnumerable.sol  
  
art/  
  BokkyPooBahsDateTimeLibrary.sol  
  VeArtProxy.sol  
  
bridge/  
  hyperlane/HyperlaneAdapter.sol  
  LeafMessageOrchestrator.sol  
  MessageOrchestrator.sol  
  RootLocalAdapter.sol  
  RootMessageOrchestrator.sol  
  
core/  
  VotingEscrow.sol  
  
factories/  
  FactoryRegistry.sol  
  GaugeFactory.sol  
  PoolFactory.sol  
  StablePoolFactory.sol  
  VolatilePoolFactory.sol  
  
fees/  
  CustomFeeModule.sol  
  DiscountRegistry.sol  
  FlatFeeQuoter.sol  
  PriorityFeeMevTaxModule.sol
```

```
gauge-creation/  
  DefaultGaugeCreationModule.sol  
  GaugeManager.sol  
  GovernanceCreationModule.sol  
  
gauges/  
  Gauge.sol  
  V2Gauge.sol  
  
handlers/  
  LeafEmissionsHandler.sol  
  RootEmissionsHandler.sol  
  
hooks/dynamic/  
  DynamicSwapFeeHook.sol  
  libraries/TransientMevTaxLib.sol  
  
libraries/  
  AllocationLogicLibrary.sol  
  BalanceLogicLibrary.sol  
  CheckpointLogicLibrary.sol  
  DelegationLogicLibrary.sol  
  LeafAllocationLibrary.sol  
  ParamsLib.sol  
  PoolOracle.sol  
  ProtocolConstants.sol  
  RewardsLogicLibrary.sol  
  Roles.sol  
  SafeCastLibrary.sol  
  VelodromeTimeLibrary.sol  
  VolatilityRingLibrary.sol  
  VotingRewardsFactoryLibrary.sol  
  
metarouter/  
  Metarouter.sol  
  TransientTracking.sol  
  libraries/ClaimsLib.sol  
  libraries/ClPositionLib.sol  
  libraries/Commands.sol  
  libraries/CrosschainLib.sol  
  libraries/FundsLib.sol  
  libraries/GaugeLib.sol  
  libraries/LiquidityLib.sol  
  libraries/MetarouterState.sol  
  libraries/PaymentsLib.sol  
  libraries/StakeRelayLib.sol  
  libraries/StakingLib.sol  
  
migration/  
  AerodromeMigration.sol  
  Migration.sol  
  VelodromeMigration.sol  
  VelodromeMigrationEntrypoint.sol  
  
minter/  
  Minter.sol
```

```
pools/  
  Pool.sol  
  PoolFactoryIndexation.sol  
  PoolFees.sol  
  StablePool.sol  
  VolatilePool.sol  
  tape/BasePoolTape.sol  
  tape/CLPoolTape.sol  
  tape/PoolTape.sol  
  
relay/  
  MaxiRelay.sol  
  ProtocolRelay.sol  
  RelayBase.sol  
  RelayFactory.sol  
  RelayRoles.sol  
  RelayToken.sol  
  RelayTokenVotes.sol  
  RelayVoteAdapter.sol  
  
relay/entrypoints/  
  BaseEntrypoint.sol  
  Compounder.sol  
  MultiConverter.sol  
  MultiEntrypoint.sol  
  MultiHybrid.sol  
  SingleConverter.sol  
  SingleHybrid.sol  
  
relay/libraries/  
  AllocationLib.sol  
  DenseQueue.sol  
  QueueLib.sol  
  RelayConfigLib.sol  
  RelayGovernanceLib.sol  
  RelayInitLib.sol  
  RelayRewardsLib.sol  
  
rewards/  
  FeeDistribution.sol  
  IncentiveStreaming.sol  
  VotingCheckpoints.sol  
  VotingRewardsFactory.sol  
  VotingRewardsManager.sol  
  
splitter/  
  Splitter.sol  
  
token/  
  BaseToken.sol  
  ReceiptToken.sol  
  Token.sol  
  
tokenRegistry/  
  TokenNFT.sol  
  TokenRegistry.sol
```

```
voter/  
  LeafVoter.sol  
  LeafVoterStorageBase.sol  
  LeafVoterStorageLayout.sol  
  Voter.sol  
  VoterStorageBase.sol  
  VoterStorageLayout.sol  
  
vpm/  
  VoterPaymentsModule.sol  
  
metadex-slipstream/contracts/  
  
core/  
  CLFactory.sol  
  CLPool.sol  
  
core/fees/  
  CustomUnstakedFeeModule.sol  
  
core/indexation/  
  CLFactoryIndexation.sol  
  
core/libraries/  
  BitMath.sol  
  FixedPoint128.sol  
  FixedPoint96.sol  
  FullMath.sol  
  LiquidityMath.sol  
  LowGasSafeMath.sol  
  Oracle.sol  
  Position.sol  
  SafeCast.sol  
  SqrtPriceMath.sol  
  SwapMath.sol  
  Tick.sol  
  TickBitmap.sol  
  TickMath.sol  
  TransferHelper.sol  
  UnsafeMath.sol  
  hook/SwapHookLib.sol  
  
gauge/  
  CLGauge.sol  
  CLGaugeFactory.sol  
  Gauge.sol  
  
gauge/libraries/  
  Roles.sol  
  
libraries/  
  EnumerableSet.sol  
  ProtocolConstants.sol  
  VelodromeTimeLibrary.sol  
  
periphery/base/
```

```

BlockTimestamp.sol
ERC721Permit.sol
LiquidityManagement.sol
Multicall.sol
PeripheryImmutableState.sol
PeripheryPayments.sol
PeripheryPaymentsWithFee.sol
PeripheryValidation.sol
SelfPermit.sol

periphery/lens/
  CLInterfaceMulticall.sol
  Metaquoter.sol
  MixedRouteQuoterV1.sol
  MixedRouteQuoterV2.sol
  MixedRouteQuoterV3.sol
  Quoter.sol
  QuoterV2.sol
  TickLens.sol

periphery/libraries/
  BytesLib.sol
  CallbackValidation.sol
  ChainId.sol
  HexStrings.sol
  LiquidityAmounts.sol
  MetaquoterLib.sol
  NFTDescriptor.sol
  NFTSVG.sol
  OracleLibrary.sol
  Path.sol
  PoolAddress.sol
  PoolTicksCounter.sol
  PositionKey.sol
  PositionValue.sol
  SqrtPriceMathPartial.sol
  TokenRatioSortOrder.sol
  TransferHelper.sol

periphery/
  LpMigrator.sol
  NonfungiblePositionManager.sol
  NonfungibleTokenPositionDescriptor.sol
  SwapRouter.sol

```

2.2 Summary of Findings

Each finding from the audit has been assigned a severity level of “Critical”, “High”, “Medium”, “Low” or “Informational”. These severities aim to capture the impact and likelihood of each potential issue.

3 Findings

3.1 High Severity Findings

3.1.1 Try/catch blocks can be made to intentionally fail

Description: In previous versions of the codebase, cross-chain messages arriving on leaf chains included downstream calls wrapped in `try/catch` blocks, including calls to `LeafVoter.notifyGaugeRate()` and `LeafVoter.checkpointReward()`. A downstream revert could therefore be caught without reverting state changes already made while processing the message.

This enabled the following attack pattern:

1. Before relaying a valid pending message, an attacker enters a `nonReentrant` function on a downstream contract that yields control flow through a callback.
2. From within the callback, the attacker relays the pending cross-chain message.
3. Leaf message processing calls back into the same contract, where the active reentrancy guard causes the downstream call to revert.
4. The surrounding `try/catch` catches the revert, allowing message processing to complete while its nonce and any earlier allocation changes remain committed.

The objective is not conventional reentrancy, but rather to deliberately trigger a revert that the protocol treats as a downstream failure. This was particularly problematic for `notifyGaugeRate()`, where the failed operation was not retryable and the corresponding information could therefore be permanently lost.

Status: This was addressed as part of the cross-chain messaging refactor in [PR 404](#).

The team had also identified a separate but related concern: if an attacker could cause a leaf-chain message to fail permanently, a deallocation could release voting power on the root chain without removing the corresponding allocation on the leaf, allowing the same voting power to be reused.

Under the new design, root-chain voting power is not released until the leaf successfully processes the deallocation and returns a confirmation message. A failed leaf message therefore leaves the voting power unavailable rather than duplicating it.

One remaining instance of this deliberate `try/catch` failure behavior existed in the `claimRewards()` flow, where failures from registered `VotingRewardsManager` contracts could be caught while the inbound message was still consumed. This is addressed by [PR 583](#), which allows these reverts to bubble so the message can be retried.

3.1.2 Dispatch `refundRecipient` enables reentrancy concerns

Description: Dispatching a cross-chain message from the root chain can yield control flow to the caller-supplied `_refundRecipient` during the dispatch. Although the root voter is protected by `nonReentrant`, the system consists of several interconnected contracts, so cross-contract reentrancy remains a concern. Some functions, such as `processDeallocation()`, are also intentionally not protected because they may naturally be called in a reentrant context during local root dispatches.

In certain intermediate versions of the code, this created a concrete issue. For example, `allocate()` dispatched the chain-allocation messages before constructing the gauge-allocation messages, causing the latter to re-read the position's live stake after control flow had already been yielded. An attacker could increase their stake through the refund callback after the chain allocation had been recorded but before the gauge payload was constructed. Because increasing the stake amount did not change the stake end, the existing stale-shape check did not detect the modification. The resulting gauge message could therefore describe more voting power than the corresponding chain allocation had booked.

Status: This concern was addressed in [PR 404](#). CEI is now respected around dispatches, with state updates and message construction completed before dispatching at the end of the function.

For example, allocation entrypoints now snapshot the live stake and emissions rate once, build all payloads from that snapshot, and only then dispatch them.

3.2 Medium Severity Findings

3.2.1 Unrelated Relay balance changes could satisfy swap-output checks

Description: Relay entrypoints previously measured swap output by reading the Relay's token balance before and after `MetaRouter.execute()` and treating the difference as the amount produced by the swap.

However, the Relay's balance could change during router execution for reasons unrelated to the swap. For example, a malicious token in the swap route could gain control flow and trigger `claimFees()`, causing tokens to enter the Relay. That unrelated inflow would be counted as swap output, potentially satisfying `minAmountOut` even if the swap itself returned less than required. The extra tokens would also be passed into the subsequent compound or reward-notification flow as though they were swap proceeds.

Conversely, an unrelated outflow during execution could cause an otherwise valid swap to fail its minimum-output check.

Status: This was addressed in [PR 478](#). Swap output is now sent to the entrypoint and measured using the entrypoint's own balance before being forwarded to the Relay. Since no other protocol flow credits the entrypoint, unrelated Relay balance changes can no longer affect the measured swap output.

3.2.2 Failed balance reads could corrupt balance-diff accounting

Description: Solady's `SafeTransferLib` includes a `balanceOf()` helper that returns zero if the underlying `static-call` fails rather than reverting. While useful for some compatibility-oriented checks, this behavior is unsafe when two balance reads are subtracted to determine how much value was received.

For example, if the pre-transfer balance read failed and returned zero while the post-transfer read succeeded, the computed increase would include the contract's entire pre-existing token balance. Conversely, a failed post-transfer read could cause an underflow or incorrectly report no output. A token capable of selectively failing balance reads could therefore manipulate accounting based on the resulting balance difference.

This was particularly relevant to the Relay and its entrypoints, where several flows used token balance diffs.

Status: MetaDEX was changed to use standard `IERC20.balanceOf()` calls for balance-diff accounting instead of Solady's `SafeTransferLib.balanceOf()` helper. A failed balance read therefore reverts rather than being interpreted as a zero balance.

3.2.3 Per-batch reward rounding could leave the Relay unable to pay claims

Description: `RelayRewardsLib.notifyReward()` previously advanced the reward index using:

```
indexDelta = floor(amount * ACC_SCALE / supply)
```

It then increased `accountedBalance` using another rounded-down calculation:

```
distributed = floor(indexDelta * supply / ACC_SCALE)
```

Claims, however, calculate a holder's rewards using a single floor over the cumulative index movement since their last checkpoint. As a result, a holder settling several notifications at once could be entitled to more than the sum of the separately rounded-down distributed amounts.

For example, with `ACC_SCALE = 10`, supply 3, and two notifications of 2:

- Each notification increases the index by 6, but adds only 1 to `accountedBalance`.
- After both notifications, the index is 12 and `accountedBalance` is 2.
- A holder owning the full supply can claim $\text{floor}(3 * 12 / 10) = 3$.

The claim would then underflow when subtracting 3 from an `accountedBalance` of 2, preventing the claim even though the Relay holds sufficient tokens.

Status: This was addressed in [PR 478](#) by rounding the amount added to `accountedBalance` upward.

3.3 Low Severity Findings

3.3.1 Permissionless index updates could delay migration fee flushing

Description: The V2 migration process requires gauges to be temporarily revived so that fees can be flushed from each pool and gauge into its `FeesVotingReward`. The intended sequence is to kill the gauges, use the `MIGRATION_TOKEN_ID` to atomically revive, vote, and kill them in epoch N, then atomically revive, call `distribute()`, and kill them again in epoch N+1.

However, both `distribute()` and `updateFor()` are permissionless. Before the epoch N+1 migration batch executes, an attacker could call either function while the gauge is still killed. This can consume the gauge's share of the weekly index growth without performing the fee flush, which requires the gauge to be alive.

The later `revive`, `distribute()`, and `kill` batch may then have no remaining index growth to process and become a no-op. The migration would need to wait for the next epoch, and the same action could be repeated to continue delaying the fee flush.

Status: This will be accounted for in the migration runbook. Executing the batch as one of the first transactions of the epoch reduces the opportunity for interference.

3.3.2 Withdrawn token IDs could not be revived

Description: Withdrawing an expired `VotingEscrow` position does not burn its NFT. Instead, the token ID remains as an empty shell with an amount and end timestamp of zero, while its corresponding `Voter` state is cleared. The code intended for these token IDs to be reusable, but in an older version there was no valid path to revive them:

- `increaseStakeAmount()` rejected the position because its end timestamp had expired.
- `increaseStakingPeriod()` reached `Voter.parkOnChain0()`, which rejected the zero stake.
- `upgradeToPermanentStake()` also rejected the expired position.

As a result, a withdrawn token ID remained permanently unusable despite the intended revival behavior.

Status: This was addressed in [PR 514](#) by adding a dedicated `reviveStake()` entrypoint.

3.3.3 Gauge fee tokens could differ from the underlying pool tokens

Description: Earlier versions of the gauge-creation system allowed a registered creation module to provide the reward-token array used to initialize a pool's `VotingRewardsManager`. The manager treated the first two entries as the gauge's fee tokens, while the gauge itself collected the actual `token0` and `token1` from its pool.

The permissionless creation path derived these values from the pool and was therefore safe. The governance creation path, however, could supply arbitrary values. Because the gauge-to-rewards-manager relationship is fixed at creation, this could permanently break fee accounting for the gauge.

Status: This was addressed as part of the gauge-creation work in [PR 385](#) and [PR 387](#). `GaugeFactory` now derives the two fee tokens directly from `IPool(target).token0()` and `token1()` when creating the `VotingRewardsManager`, rather than trusting values supplied by the creation module.

3.3.4 Relay reward configuration could disable reward processing

Description: In a previous version of the code, a Relay could be initialized without either a `COMPOUNDER` or `CONVERTER`. Without one of these entrypoints, incoming fee and incentive rewards could not be processed and would remain as unaccounted token balances in the Relay.

For Maxi and Protocol L1 Relays, these roles are fixed at initialization, so a bad configuration could not be corrected later through normal role management.

Status: This was addressed in [PR 529](#). Maxi and Protocol L1 Relays now revert with `MissingEntrypoint` unless at least one compounder or converter is configured. Protocol L2 Relays are exempt because entrypoints can be attached later through the timelocked configuration flow.

3.3.5 Invalid module return data could still revert pool swaps

Description: Pool factories use `excessivelySafeStaticCall` when querying fee modules so that a misconfigured or malicious module cannot make pools unusable. The call limits both the gas forwarded to the module and the amount of return data copied by the caller.

However, earlier implementations did not fully validate the returned data before decoding it. For example, the V2 `PoolFactory` decoded a successful fee-module response directly as a `uint24`:

```
(bool success, bytes memory data) =
    feeModule.excessivelySafeStaticCall(500_000, 32, abi.encodeWithSelector(IFeeModule.getFee.
        selector, _pool));

if (success) {
    uint256 customFee = abi.decode(data, (uint24));
    ...
}
```

A module could return a 32-byte value larger than `type(uint24).max`, causing `abi.decode()` to revert. Similarly, returning too little data could also cause decoding to revert.

Status: This was addressed in [PR 329](#). Module responses are now checked for the exact expected length before decoding.